

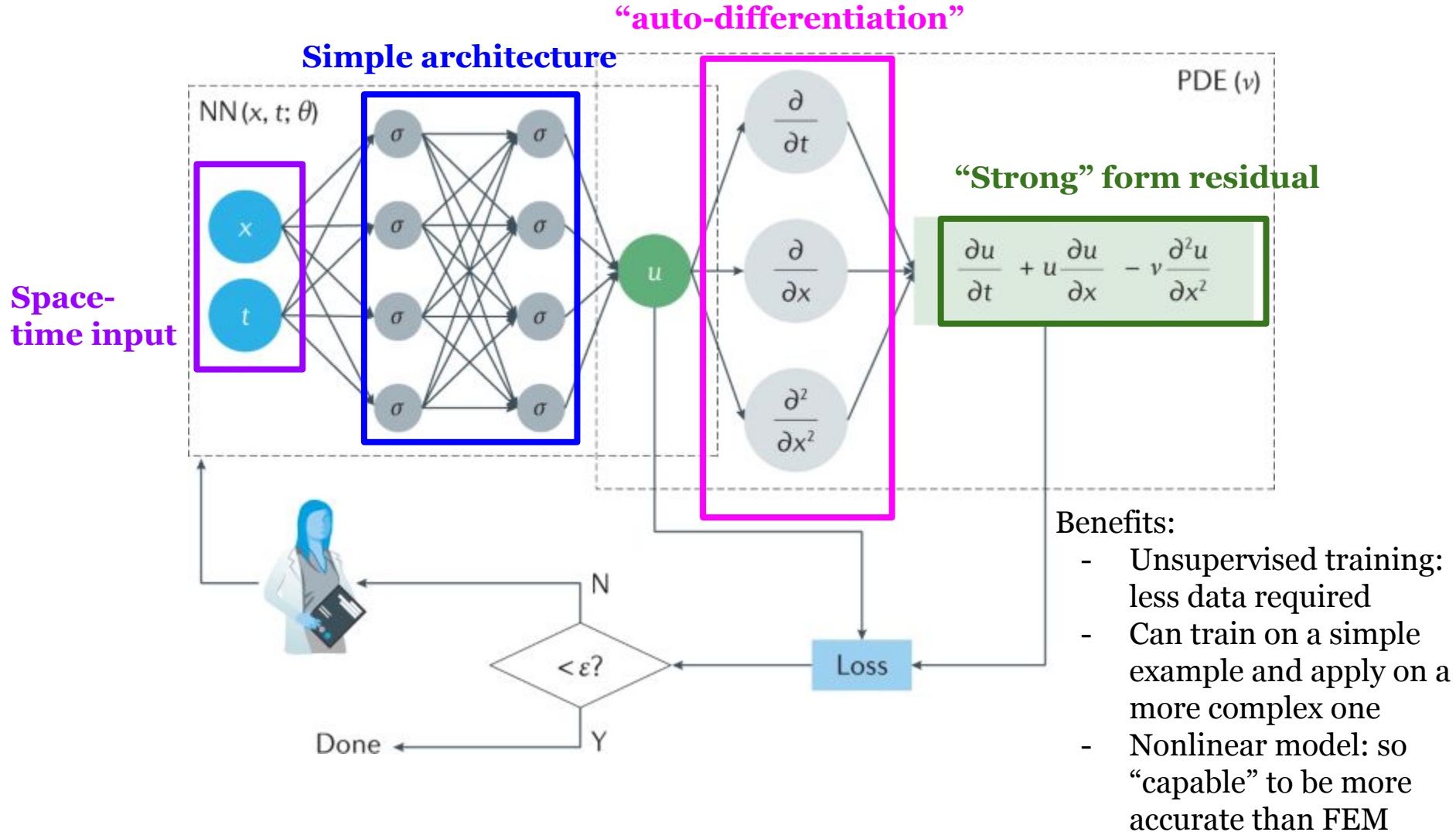
Context

1. Numerical solvers like FEM and FVM are expensive and must be run for every new configuration
2. Neural networks can be trained ahead of time and if done well, the actual use is instantaneous allowing quick design and control implementations
3. These data-driven models rely on a lot of pre-generated data to learn physics in a pretty much “black box” way
4. Adding physics into training can help reduce reliance on massive data libraries and produce more generalizable models
5. Typical neural networks don’t generalize well to complex geometries (allegedly) - hence we are interested in graph neural networks which integrate irregular meshes into their architecture

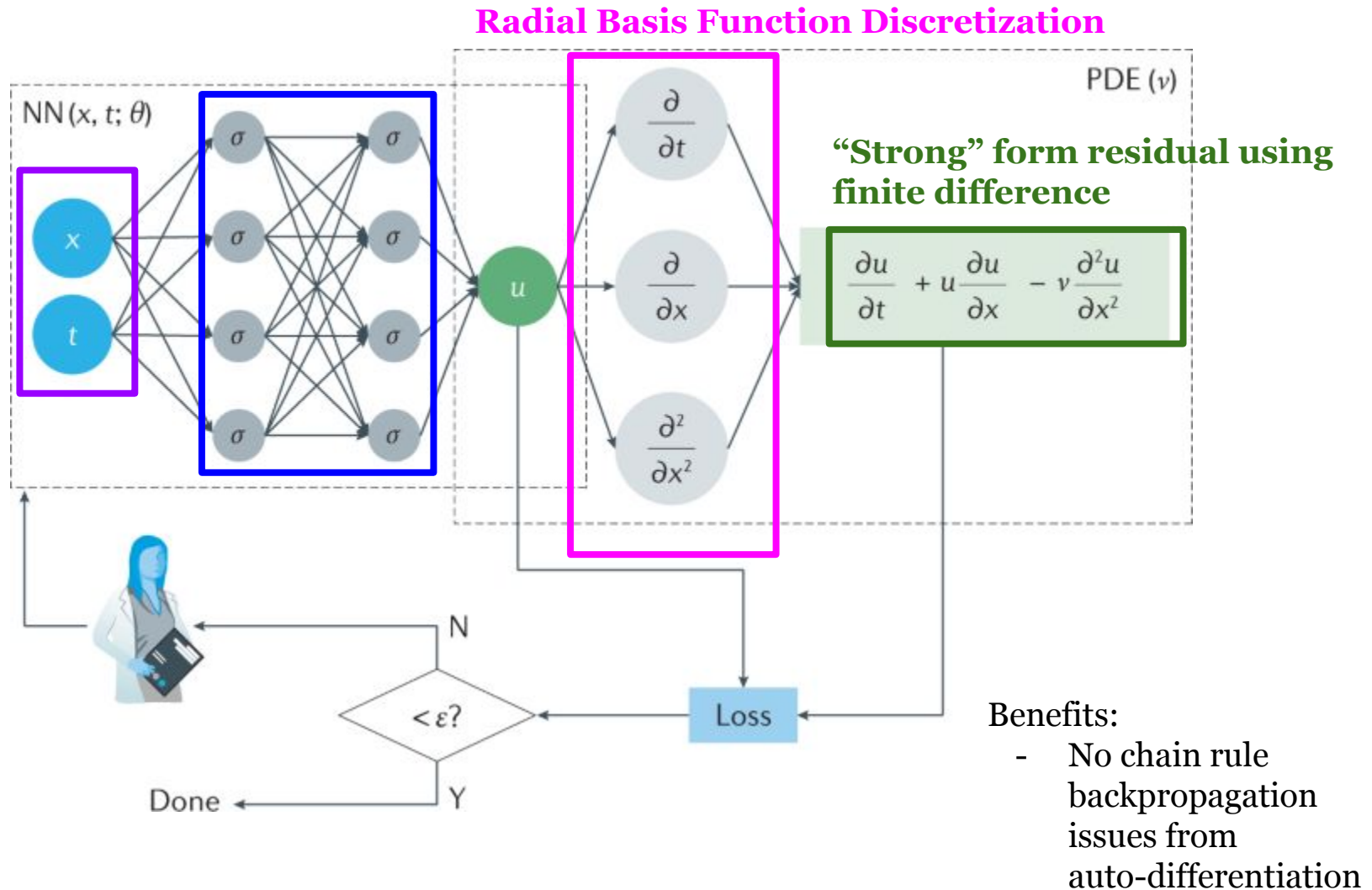
So here we are - we are interested in physics-informed graph neural networks - rest of slides is essentially just more details of the current work and some future steps at the end - all and any ideas are welcome and frankly this all does not require much advanced machine learning knowledge as it is mostly just tinkering

Sponsoring company is Mathworks, the MATLAB people, so part of “novelty” here is actually also showcasing MATLAB capabilities to do this sort of work since Python usually dominates the space

Continuous physics-informed neural network



Discretized physics-informed neural network

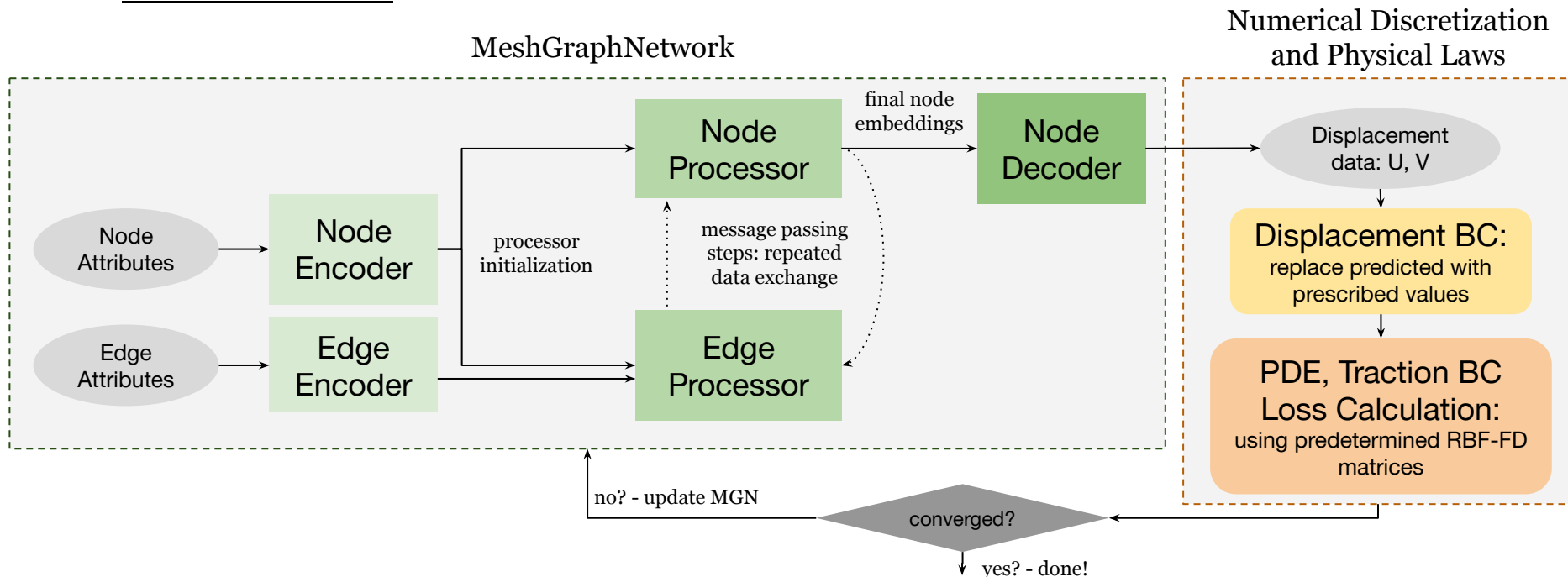


1. Overview graph physics-informed neural network

1. Features and mesh inputs:

- a. Node attributes: [x, y, hot-node vectors]
- b. Edge attributes (bidirectional edges): [x width, y width, length magnitude]
- c. Edge index: [node sender row, node receiver row]
 - i. Used inside edge processor to combine sender & receiver node embeddings with respective edge embeddings
- d. Edge-node connectivity matrix: ($\#$ edges) $\times$ ($\#$ nodes) [adjacency matrix]
 - i. Used inside node processor to aggregate neighboring edge embeddings per node
- e. Node mask and edge mask
 - i. More relevant for multi-batch padded data

2. PI-MGN Schematic:



2. Numerical discretization and physical laws

1. Governing equations: traction & displacement BCs are defined individually in X & Y directions

<u>Boundary conditions (BC)</u>	<u>Boundary notation</u>	<u>Equilibrium equations</u>	<u>Constitutive law</u>	
$\sigma_{xx} \cdot n_x + \sigma_{xy} \cdot n_y = t_x$	$(x, y) \in \Gamma_{t_x}$	$\Gamma_{t_x} \cap \Gamma_{d_x} = 0$	$\frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} = 0 \quad (x, y) \in \Omega$	$\sigma_{xx} = 2\mu \frac{\partial u}{\partial x} + \lambda \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right)$
$\sigma_{xy} \cdot n_x + \sigma_{yy} \cdot n_y = t_y$	$(x, y) \in \Gamma_{t_y}$	$\Gamma_{t_y} \cap \Gamma_{d_y} = 0$	$\frac{\partial \sigma_{xy}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} = 0 \quad (x, y) \in \Omega$	$\sigma_{xy} = \mu \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \right)$
$u - u_0 = 0$	$(x, y) \in \Gamma_{d_x}$	$\Gamma_{t_x} \cup \Gamma_{d_x} = \Gamma$		$\sigma_{yy} = 2\mu \frac{\partial v}{\partial y} + \lambda \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right)$
$v - v_0 = 0$	$(x, y) \in \Gamma_{d_y}$	$\Gamma_{t_y} \cup \Gamma_{d_y} = \Gamma$		

2. Strong form discretized residuals: $\mathcal{L}_x := \frac{\partial}{\partial x} \quad \mathcal{L}_y := \frac{\partial}{\partial y} \quad \mathcal{L}_{xx} := \frac{\partial^2}{\partial x^2} \quad \mathcal{L}_{yy} := \frac{\partial^2}{\partial y^2} \quad \mathcal{L}_{xy} := \frac{\partial^2}{\partial x \partial y}$

Traction (BCs)

$$\begin{bmatrix} i_{\text{mask}}^{t_x} \\ i_{\text{mask}}^{t_y} \end{bmatrix} \cdot \begin{bmatrix} n_x \cdot (2\mu + \lambda) \mathcal{L}_x + n_y \cdot \mu \mathcal{L}_y & n_y \cdot \mu \mathcal{L}_x + n_x \cdot \lambda \mathcal{L}_y \\ n_x \cdot \mu \mathcal{L}_y + n_y \cdot \lambda \mathcal{L}_x & n_y \cdot (2\mu + \lambda) \mathcal{L}_y + n_x \cdot \mu \mathcal{L}_x \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} - \begin{bmatrix} t_x \\ t_y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

*Derivative operators interpolated via radial basis functions shown on the next slide

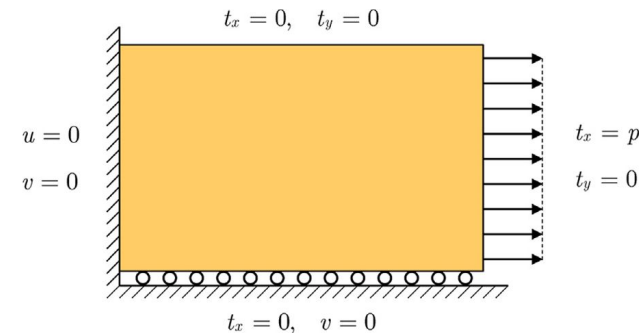
Equilibrium equations

$$\begin{bmatrix} \mu \mathcal{L}_{yy} + (2\mu + \lambda) \mathcal{L}_{xx} & (\mu + \lambda) \mathcal{L}_{xy} \\ (\mu + \lambda) \mathcal{L}_{xy} & \mu \mathcal{L}_{xx} + (2\mu + \lambda) \mathcal{L}_{yy} \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Displacement (BCs)

$$\begin{bmatrix} i_{\text{mask}}^{d_x} \\ i_{\text{mask}}^{d_y} \end{bmatrix} \cdot \left(\begin{bmatrix} u \\ v \end{bmatrix} - \begin{bmatrix} u_0 \\ v_0 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Example of mixed boundary conditions:

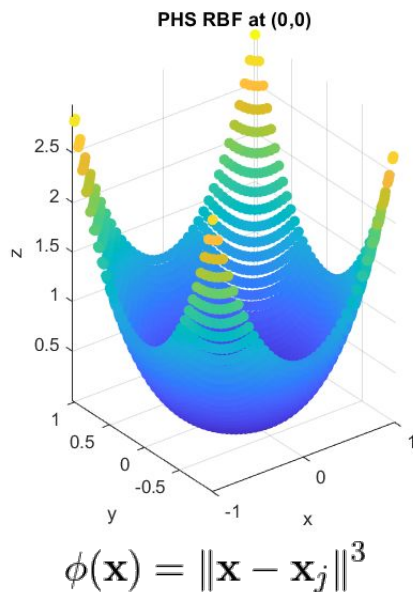


3. Numerical discretization: RBF-FD

1. Solution interpolation using radial basis functions (polyharmonic splines here) supported by monomials centered at each interpolation point and its stencil:

$$s(\mathbf{x}) = \lambda_1 \phi(\|\mathbf{x} - \mathbf{x}_1\|) + \dots + \lambda_n \phi(\|\mathbf{x} - \mathbf{x}_n\|) + \mu_1 + \mu_2 x + \mu_3 y + \mu_4 x^2 + \mu_5 xy + \mu_6 y^2 + \dots + \mu_k p_k(\mathbf{x})$$

$$s(\mathbf{x}) = \sum_{j=1}^n \lambda_j \phi(\|\mathbf{x} - \mathbf{x}_j\|) + \sum_{j=1}^k \mu_j p_j(\mathbf{x})$$



Polynomials p_j have degree of $D_m: k$ is determined by $D_m + \dim$ choose $\dim$

Other variables are:

n : stencil size

λ : interpolation coefficients

μ : polynomial weights

Extra constraint: to make square matrix

$$\sum_{j=1}^n \lambda_j p_i(\mathbf{x}_j) = 0, \quad i = 1, 2, \dots, k$$

2. Vectorize solution interpolation for every stencil, and re-formulate to solve for interpolation weights (λ and μ) as a function of the nodal solutions $\mathbf{f}$:

$$\begin{bmatrix} A & P \\ P^T & O \end{bmatrix} \begin{bmatrix} \lambda \\ \mu \end{bmatrix} = \begin{bmatrix} \underline{f} \\ \underline{0} \end{bmatrix} \Rightarrow \begin{bmatrix} \lambda \\ \mu \end{bmatrix} = \begin{bmatrix} A & P \\ P^T & O \end{bmatrix}^{-1} \begin{bmatrix} \underline{f} \\ \underline{0} \end{bmatrix}$$

3. Derivative of the solution interpolation for every stencil is formulated,

$$Ls(\mathbf{x})|_{\mathbf{x}=\mathbf{x}_e} = \sum_{j=1}^n \lambda_j L\phi(\|\mathbf{x} - \mathbf{x}_j\|)|_{\mathbf{x}=\mathbf{x}_e} + \sum_{j=1}^k \mu_j Lp_j(\mathbf{x})|_{\mathbf{x}=\mathbf{x}_e}$$

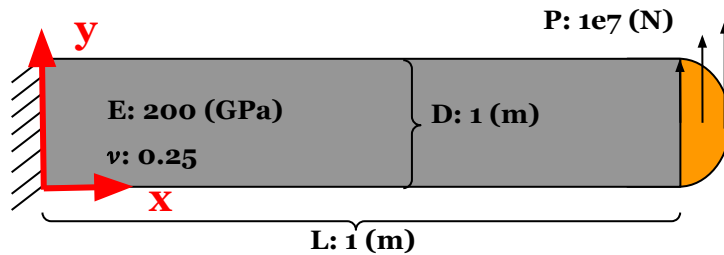
... vectorized, and combined with interpolation weights to solve for full derivative weights $\mathcal{L}$:

$$= \begin{bmatrix} \underline{b} & \underline{c} \end{bmatrix} \begin{bmatrix} \lambda \\ \mu \end{bmatrix} = \left(\begin{bmatrix} \underline{b} & \underline{c} \end{bmatrix} \begin{bmatrix} A & P \\ P^T & O \end{bmatrix}^{-1} \right) \begin{bmatrix} \underline{f} \\ \underline{0} \end{bmatrix}$$

Global $\mathcal{L}$: rows correspond to separate solutions $\mathbf{f}$, columns are respective stencils of that solution $\mathbf{f}$.

4. Case study: Timoshenko cantilever beam

1. Schematic:



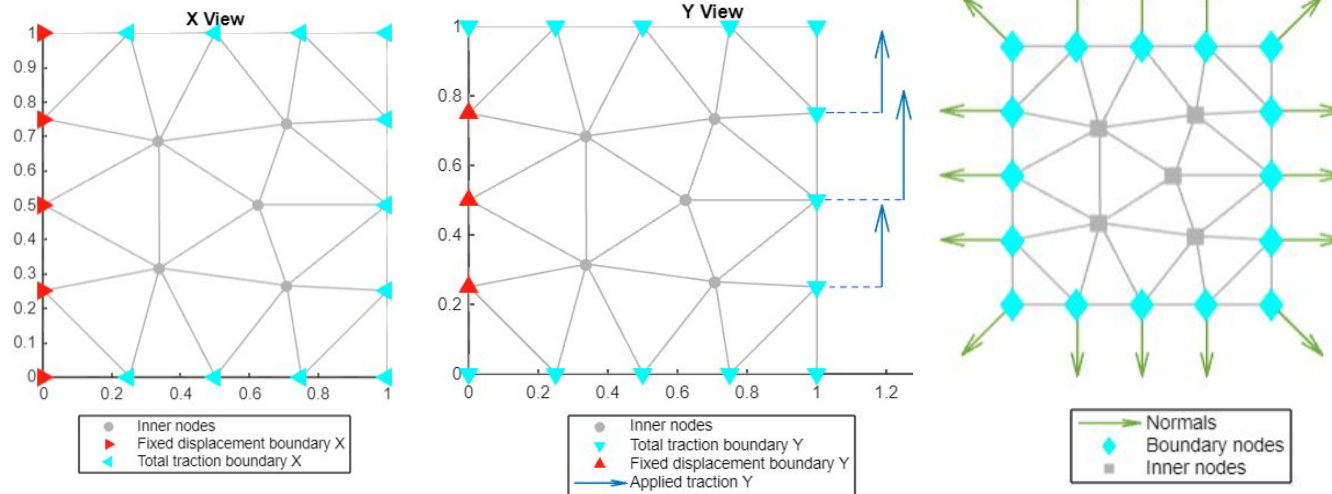
2. Exact solution:

$$\tau_{xy} = \frac{P}{2I} (Dy - y^2)$$

$$u = -\frac{P}{6EI} (y - D/2) [(6L - 3x)x + (2 + \nu)(y^2 - Dy)]$$

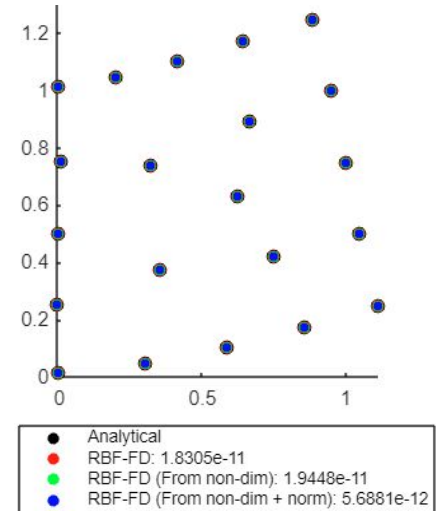
$$v = \frac{P}{6EI} [3\nu(y - D/2)^2(L - x) + (4 + 5\nu)D^2x/4 + (3L - x)x^2]$$

3. Mesh and index definitions: 21 nodes, 88 edges, left corners not fixed in y direction



4. Validation:

4.1 Solve RBF-FD



4.2 Calculate residuals from analytical solution using RBF-FD matrices:

Max residual: 1.1689e-10
Min residual: 1.2802e-11

Max residual non-dimensional: 1.6215e-25
Min residual non-dimensional: 1.8145e-26

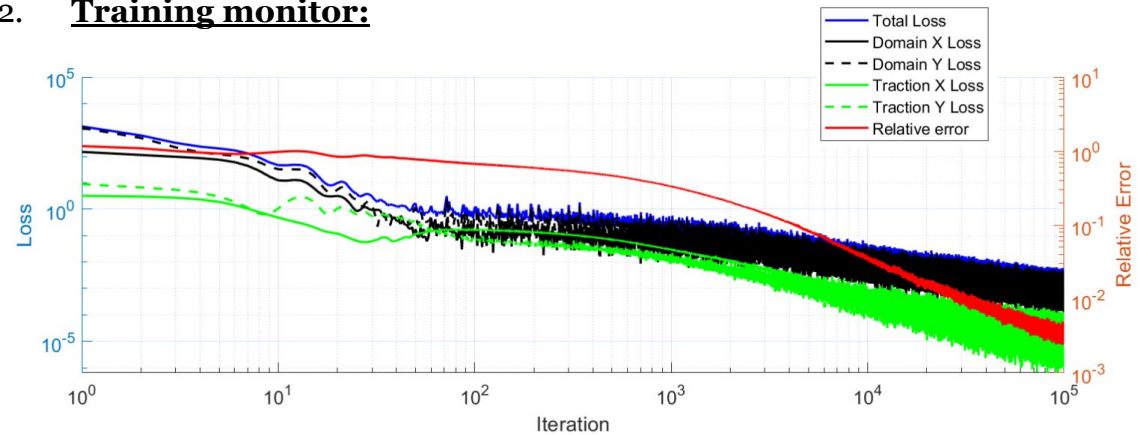
Max residual non-dimensional + normalized: 1.0452e-25
Min residual non-dimensional + normalized: 4.4412e-27

Case study: solid mechanics - e. Timoshenko beam (ADAM)

1. Training and MGN parameters

1. Initial learning rate: $1e-4$
2. Learning rate decay: $5e-5$
3. Gradient decay: 0.8
4. Square gradient decay: 0.8
5. Final layer normalization: encoder and processing layers
6. Activation function: "swish"
7. Residual calculation addition layer in the processors
8. Edges averaged instead of summed
9. Message passing steps: 5
10. Hidden layers: 2
11. Hidden neurons: 32

2. Training monitor:

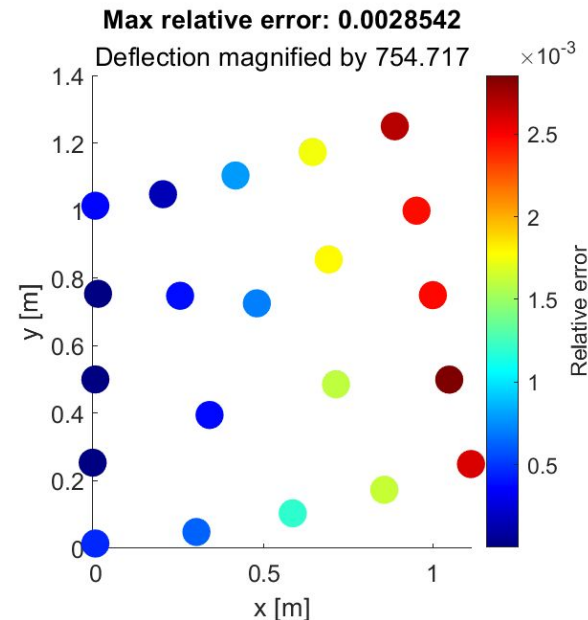
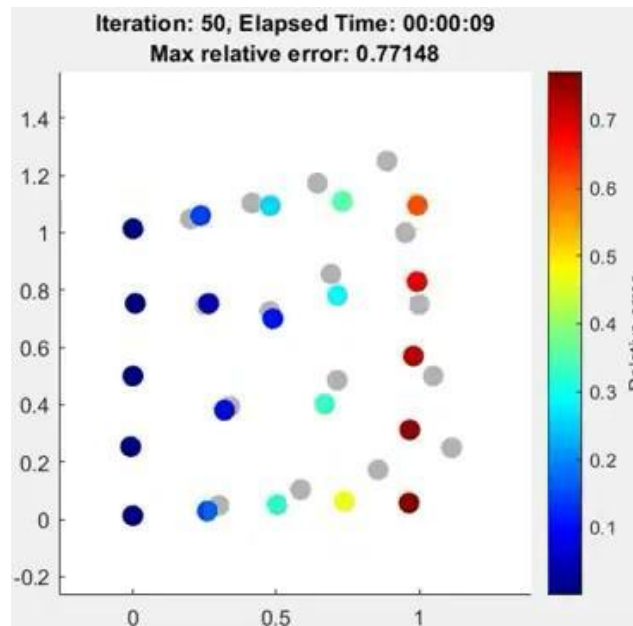


3. Final results:

Grey points: exact solution

Error <1% reached within ~30k iterations and ~1hour

Reducing # of nodes significantly improves speed and accuracy of convergence!

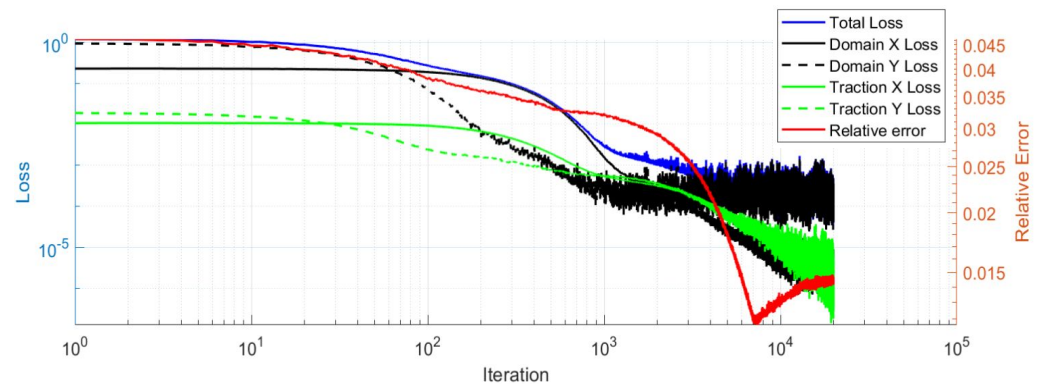


Case study: solid mechanics - e. Transfer learning (ADAM)

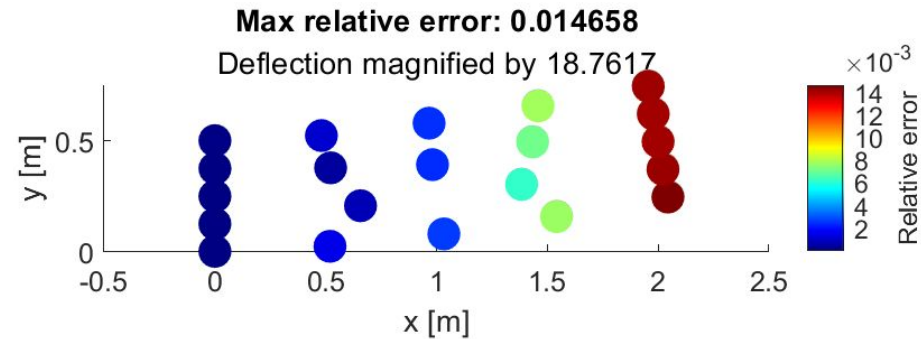
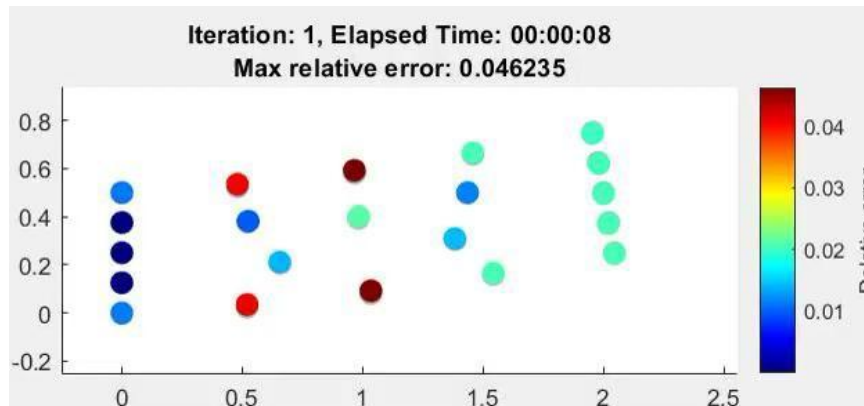
1. Training and MGN parameters

1. Initial learning rate: $1e-6$
2. Learning rate decay: $5e-7$
3. Gradient decay: 0.95
4. Square gradient decay: 0.999
5. Final layer normalization: encoder and processing layers
6. Activation function: "swish"
7. Residual calculation addition layer in the processors
8. Edges averaged instead of summed
9. Message passing steps: 5
10. Hidden layers: 2
11. Hidden neurons: 32

2. Training monitor:



3. Final results:



Original square geometry has been scaled into a 2m by 0.5 m beam and transfer learning solution performed. Even before training starter, initial error was <5% using model trained on a square! Note that convergence is heavily hyperparameter dependent, from general tinkering higher initial learning rate and lower ADAM gradient weights generally improve learning cost.

General next steps and current work

- Generalization steps
 - Geometry generalization
 - Simple body load and alternating Dirichlet boundary conditions (changing how geometry is fixed to space)
 - Goal: Can we generate a random shape, randomly fix it to space, and model how it will deflect without need of FEM or complex discretization?
 - Boundary condition generalization
 - See Timoshenko beam as example
 - Goal: Can we change inputs into features and generalize to different tractions altogether?
 - Also interesting but less priority now: Extension to 3D
- Some questions to answer to validate the model (as in is it actually better?)
 - Compare with simpler auto-derivative code if RBF-FD makes significant improvement
 - Compare RBF-FD and FEM (that one uses weak form and numerical integration of weak form of governing equations - more stable but more annoying to implement and less flexible on geometry)
 - Is adding data loss helpful or not here - supervised, semi-supervised, or unsupervised learning is best for most generalizable results?
- Other interesting questions and points to consider to improve the model:
 - Mesh smoothing options: How to reduce un-physical and parasitic deflections during training like self-intersections, negative-volumes, strange behaviors, trivial solutions, locking?